Understanding Templates in C++

Templates are one of the most powerful features of C++. They allow you to write **generic and reusable code** that works with **any data type**. Instead of duplicating code for different data types (like `int`, `float`, `double`, etc.), templates let you define a blueprint, which the compiler uses to generate type-specific code automatically.

What Are Templates in C++?

Templates are a feature of C++ that supports **generic programming**—a programming paradigm where algorithms are written in terms of **types to be specified later**. There are two main types of templates:

- Function Templates
- Class Templates

---

1. Function Templates

What Are Function Templates?

A **function template** defines a function in a generic way. It works for any data type and generates a new version of the function based on the type of arguments passed.

Syntax:

```
template <typename T>
T functionName(T arg1, T arg2)
{
    // Function logic
}
```

You can also use `class` instead of `typename` (both are valid):

```
template <class T>
```

Example: A Generic Function to Find Maximum

```
#include <iostream>
using namespace std;

template <typename T>
T findMax(T a, T b)
{
    if(a > b)
        return a;
    else
        return b;
}
int main()
{
    cout << "Max of 3 and 7: " << findMax(3, 7) << endl;
    cout << "Max of 4.5 and 2.3: " << findMax(4.5, 2.3) << endl;
    cout << "Max of 'A' and 'Z': " << findMax('A', 'Z') << endl;
    return 0;
}
```

Output:

```
Max of 3 and 7: 7
Max of 4.5 and 2.3: 4.5
Max of 'A' and 'Z': Z
```

Note on Type Consistency

Function templates expect **parameters of the same type**. If you pass different types, it can cause ambiguity or require an explicit cast.

---

2. Class Templates

What Are Class Templates?

A **class template** is a blueprint for creating a class that can work with **any data type**.

Syntax:

```
template <typename T>
class ClassName
{
    T member;
public:
    ClassName(T val)
    {
        member = val;
    }
    void display()
    {
        cout << "Value: " << member << endl;
    }
};
```

Example: A Generic Box Class

```
#include <iostream>
using namespace std;

template <typename T>
class Box
{
    T content;
public:
    Box(T c)
    {
        content = c;
    }
    T getContent()
    {
        return content;
```

```
    }
};

int main()
{
    Box<int> intBox(10);
    Box<string> strBox("Hello Template");

    cout << "intBox contains: " << intBox.getContent() << endl;
    cout << "strBox contains: " << strBox.getContent() << endl;

    return 0;
}
```

Output:

```
intBox contains: 10
strBox contains: Hello Template
```

---

3. Template with Multiple Parameters

Example: Pair Class

```
#include <iostream>
using namespace std;

template <typename T1, typename T2>
class Pair
{
    T1 first;
    T2 second;
public:
    Pair(T1 f, T2 s)
    {
        first = f;
        second = s;
```

```
    }
    void display()
    {
        cout << "First: " << first << ", Second: " << second << endl;
    }
};
int main()
{
    Pair<int, string> myPair(1, "One");
    myPair.display();
    return 0;
}
```

Output:

```
First: 1, Second: One
```

---

4. Template Specialization

What is Template Specialization?

Template specialization allows you to define a custom implementation of a template for a specific type.

4.1 Function Template Specialization

```
#include <iostream>
#include <cstring>
using namespace std;

template <typename T>
bool isEqual(T a, T b)
{
    return a == b;
}
```

```
template <>
bool isEqual<const char*>(const char* a, const char* b)
{
    return strcmp(a, b) == 0;
}

int main()
{
    cout << isEqual(10, 10) << endl;
    cout << isEqual(10.5, 10.5) << endl;
    cout << isEqual("abc", "abc") << endl;
    return 0;
}
```

Output:

```
1
1
1
```

4.2 Class Template Specialization

```
#include <iostream>
using namespace std;

template <typename T>
class Printer
{
public:
    void print(T data)
    {
        cout << "General: " << data << endl;
    }
};

template <>
```

```
class Printer<char*>
{
public:
    void print(char* data)
    {
        cout << "String: " << data << endl;
    }
};

int main()
{
    Printer<int> intPrinter;
    intPrinter.print(100);

    Printer<char*> strPrinter;
    strPrinter.print((char*)"Specialized Print");

    return 0;
}
```

Output:

```
General: 100
String: Specialized Print
```

---

5. Template Functions Inside Template Classes

```
#include <iostream>
using namespace std;

template <typename T>
class Calculator
{
public:
    T add(T a, T b);
};
```

```
template <typename T>
T Calculator<T>::add(T a, T b)
{
    return a + b;
}

int main()
{
    Calculator<int> calc;
    cout << "Sum: " << calc.add(5, 7) << endl;
    return 0;
}
```

Output:

```
Sum: 12
```

Complete Example with Inheritance

```
#include <iostream>
using namespace std;

// Polymorphic base class
class Shape
{
public:
    virtual double area() const = 0;
    virtual void draw() const = 0;
    virtual ~Shape() {}
};

// Derived class: Circle
class Circle : public Shape
{
    double radius;
```

```
public:
    Circle(double r) : radius(r)
    {
    }

    double area() const override
    {
        return 3.14159 * radius * radius;
    }

    void draw() const override
    {
        cout << "Drawing Circle\n";
    }
};

// Derived class: Rectangle
class Rectangle : public Shape
{
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h)
    {
    }

    double area() const override
    {
        return width * height;
    }

    void draw() const override
    {
        cout << "Drawing Rectangle\n";
    }
};
```

```
// Template class operating on any shape-like object
template <typename T>
class ShapePrinter
{
public:
    void printDetails(const T& shape)
    {
        shape.draw();
        cout << "Area: " << shape.area() << "\n";
    }
};

// Template specialization for pointer types (to use virtual functions)
template <>
class ShapePrinter<Shape*>
{
public:
    void printDetails(const Shape* shape)
    {
        shape->draw();
        cout << "Area: " << shape->area() << "\n";
    }
};

int main()
{
    Circle c(5.0);
    Rectangle r(4.0, 6.0);

    ShapePrinter<Circle> printer1;
    printer1.printDetails(c);

    ShapePrinter<Rectangle> printer2;
    printer2.printDetails(r);

    Shape* s1 = new Circle(3.0);
```

```
    Shape* s2 = new Rectangle(2.0, 8.0);

    ShapePrinter<Shape*> printer3;
    printer3.printDetails(s1);
    printer3.printDetails(s2);

    delete s1;
    delete s2;

    return 0;
}
```

Benefits of Templates in C++

- Code Reusability
- Type Safety
- Performance
- Less Code Duplication

Common Pitfalls

- Templates must be fully defined in header files.
- Ambiguity in type deduction can lead to issues.
- Increased compilation time due to code bloat.

Conclusion

Templates in C++ are a foundational feature for writing **type-agnostic**, **generic**, and **efficient** code. Understanding and mastering templates—including function templates, class templates, and their specializations—will significantly improve your C++ programming skills.